Maximum flow: solutions

26-E01 Antiparallel arc flow, net flow, and residuals

Derivation

At a, inflow3+1 equals outflow 3+1; at b, inflow1+3 equals outflow 1+3. Source net outflow and sink inflow are both4. Net $f_{ab}=3-1=2$, $f_{ba}=-2$; other directions have their given g and negative reverse values. Residuals are $c_f(a, b)=3-2=1$, $c_f(b, a)=2-(-2)=4$.

Path s→a→b→t has residual 1 on each arc. Augment1 to get $f_{sa}=4$, $f_{sb}=1$, $f_{ab}=3$, $f_{at}=1$, $f_{bt}=4$. A corresponding nonnegative flow uses $g_{ab}=3$, $g_{ba}=0$ and the other positive net values. Vertex a receives 4, sends 3+1; b receives 1+3, sends 4. Value is 5.

Conclusion and checks

Cut S={s} has capacity 4+1=5, matching the feasible flow. Reverse residual 4 is neither merely old $g_{ab}=3$ nor an erroneous negative flow; original reverse capacity and net flow jointly determine it.

26-E02 Flow and cut certify opposite bounds

Derivation

Take $f_{sa}=4$, $f_{sb}=3$, $f_{ab}=1$, $f_{at}=3$, $f_{bt}=4$. Vertex a receives 4, sends 1+3; b receives 3+1, sends 4. Every capacity holds and value is 7. Cut S={s, a, b}, T={t} has capacity 3+4=7, proving optimality.

Conclusion and checks

Endpoint totals give only maximum$\leq\min(8,7)=7$, not a matching lower bound. An internal network can force every route through a capacity 1 bottleneck while keeping those endpoint totals, yielding only 1. A feasible value 7 flow is needed to establish equality.

26-E03 An actual pseudopolynomial path family

Derivation

After k pairs, the tuple$(f_{sa}, f_{sb}, f_{ab}, f_{at}, f_{bt})$ is$(k, k,0, k, k)$, for $k=0,\ldots, C$. For $k<C$, the first path has a→b residual 1 and at least 1 elsewhere, so $\delta=1$ and the tuple becomes$(k+1, k,1, k, k+1)$. The second path uses reverse b→a residual 1 to withdraw this crossing, giving$(k+1, k+1,0, k+1, k+1)$. Each pair adds value 2. At pair C, value 2C saturates both source arcs, so exactly 2C augmentations occur.

Maximum flow: solutions (continued)

For C=2^B, the constant-size network stores its capacities in B+1 bits each, so input length is Θ(B) but the round count 2^(B+1) is exponential. The actual trace proves the lower behavior, rather than inferring it from an O(C) upper bound.

Conclusion and checks

BFS chooses s-a-t or s-b-t, each with bottleneck C, and finishes in two augmentations. Crossing a→b belongs to no shortest two-edge route. Large capacities do not increase this example's BFS round count.

26-E04 Matching flips and a Hall obstruction

Derivation

From unmatched L2, use L2→R1→L1→R2→L3→R3. Reverse steps remove L1R1, L3R2; forward steps add L2R1, L1R2, L3R3. Size becomes 3. Since there are only three right vertices, no matching can exceed3. All left vertices{L1, L2, L3} form a cover of size 3; the matching lower bound also makes it minimum.

After deleting L3R3, set A={L1, L2, L3} has N(A)={R1, R2}, so2<3 violates Hall. Three left vertices cannot have three distinct right mates. Matching{L2R1, L3R2} still has size 2, which is therefore maximum.

Conclusion and checks

Flipping an entire alternating path preserves disjoint endpoints at all internal vertices. A Hall-deficient set is a structural certificate against full coverage.

26-E05 No residual s-t path can still be unfinished

Derivation

Initialization gives f_su=2, f_ut=0, e(s, u, t)=(−2,2,0), h=(3,0,0). Residual arcs are u→s:2, u→t:1. No residual arc leaves s, so there is no residual s-to-t path. But u does not conserve flow: this is a preflow, and the feasible-flow hypothesis of max-flow min-cut fails.

Vertex u has no admissible arc; relabel it to 1. Push 1 along u→t, obtaining excess(−2,1,1) and saturating that arc. The remaining residual neighbor s has height3, so relabel u to 4. Return 1 along u→s, giving excess(−1,0,1), f_su=1, f_ut=1.

Maximum flow: solutions (continued)

Conclusion and checks

The final feasible value 1 equals cut capacity 1 for S={s, u}. Initially pushing2 from s did not create a feasible value 2 flow. Internal conservation must hold before invoking flow/cut equivalence.

26-E06 Neighbor order changes distribution, not optimal value

Derivation

Start e(a, b, c, t)=(4,3,2,0). Relabel a to 1, push 1 to t, giving(3,3,2,1), then2 to b, giving(1,5,2,1). Relabel a to 6 and return 1 to s, giving(0,5,2,1). After discharge, L=[a, b, c], heights(6,0,0), current(s, t, t).

Relabel b to 1, push 4 to t, giving(0,1,2,5), then1 to c, giving(0,0,3,5). Move b after discharge: L=[b, a, c], heights(6,1,0), current(s, c, t). Revisit empty a. Relabel c to 1 and send3 to t, giving(0,0,0,8). Move c: L=[c, b, a], heights(6,1,1), current(s, c, t). Revisit b, a and finish.

Final original flows are sa3, sb3, sc2, ab2, at 1, bc1, bt4, ct3. Vertex a receives and sends 3; b receives and sends 5; c receives and sends 3. Cut S={s, a} has capacity 3+2+2+1=8, matching the flow.

Conclusion and checks

The main trace has bc2, bt3, ct4; this trace has 1,4,3. Optimal distributions may differ while both values equal8. Current stays at the neighbor used by the push that emptied excess.

26-E07 Safety and cost of current pointers

Derivation

A zero uv residual can recover only through reverse vu push. At that moment $h(v)=h(u)+1$, so restored uv points uphill and is inadmissible. For a positive but inadmissible uv, valid integer heights exclude $h(u)>h(v)+1$, and failure of equality leaves $h(u)\leq h(v)$. With fixed u and nondecreasing v, an admissible one-level drop cannot arise.

Reset only when u itself relabels. Each u relabels O(n) times, and each interval scans at most deg(u) neighbors. Its total is $O(n\deg(u))$; summing gives $O(n\Sigma\deg)=O(nm)$.

Maximum flow: solutions (continued)

Conclusion and checks

Resetting at every discharge rescans skipped prefixes even when height is unchanged. The once-per-height-interval charge no longer applies. This does not by itself prove incorrect output, but it invalidates that O(nm) scan proof.

26-E08 Rational capacities and exact certificates

Derivation

Send1/3 along s-a-t and 1/6 along s-b-t, total1/2. Cut S={s, a} has capacity 1/6+1/3=1/2, so the flow is maximum. Scaling by 6 gives capacities sa3, at 2, sb1, bt1, and maximum3. Restoring gives 3/6=1/2.

A denominator product D has bit length at most the sum of denominator bit lengths, O(N), but value can be2^Θ(N). The scaled Ford–Fulkerson optimum can therefore be numerically huge. Finite termination is weaker than a polynomial round bound. Edmonds–Karp instead bounds each direction's critical uses by O(n), giving O(nm) rounds independently of D's value.

Conclusion and checks

Common-denominator integer arithmetic certifies exact flow/cut equality. A small floating-point discrepancy alone is insufficient. Polynomial rational-input time requires both polynomial round count and polynomial per-round bit cost.

26-E09 Complete residual networks, not just paths

With unit capacities, the first original-flow vector (sa,sb,ab,at,bt) is (1,0,1,0,1). Positive residual arcs are exactly as:1, sb:1, ba:1, at:1, tb:1. The second path s-b-a-t cancels the ab flow, producing (1,1,0,1,1). Positive residual arcs become as:1, bs:1, ab:1, ta:1, tb:1.

Both internal vertices receive and send 1. Feasible value 2 equals cut {s} capacity 2, proving optimality. Arc ab retains residual capacity 1, but s has no residual outgoing arc, so a is unreachable.

For capacities (3,2,1,2,3), each row describes the state before augmentation:

Maximum flow: solutions (continued)

Round	All positive residual arcs	d(a,b,t), path, bottleneck
1	sa3, sb2, ab1, at2, bt3	(1,1,2), s-a-t, 2
2	sa1, as2, sb2, ab1, ta2, bt3	(1,1,2), s-b-t, 2
3	sa1, as2, bs2, ab1, ta2, bt1, tb2	(1,2,3), s-a-b-t, 1
Stop	as3, bs2, ba1, ta2, tb3	(∞,∞,∞), no path

Values are 0,2,4,5. The final original-flow vector is (3,2,1,2,3). Vertex a receives 3 and sends 1+2; b receives 2+1 and sends 3. Cut {s} capacity 3+2=5 independently proves optimality.

26-E10 Escape without shared cells

Edge-disjoint routes are (3,2)→(3,3)→(3,4)→(3,5) and (2,3)→(3,3)→(4,3)→(5,3). Sending one unit on each attains 2, matching the total capacity 2 leaving the source. Both routes use z=(3,3), violating vertex disjointness.

Split every open v into v_in and v_out, with internal capacity 1. Add unit-capacity out→in arcs in both directions between open neighbors, source→start_in, and boundary_out→sink. Take S to contain s, both halves of starts u,v, and z_in. The only S→T arc is z_in→z_out, of capacity 1. The first listed route attains 1, so the split optimum is exactly 1.

With q open cells, p distinct starts, and b open boundary cells, N=2q+2. There are q internal arcs, at most 4k(k−1) movement arcs, and p+b terminal arcs, so $M \le q+4k(k-1)+p+b$. Omit obstacles; starts must be open. Remove cycles from an integral maximum flow and decompose the remainder into unit source–sink paths. Internal capacity 1 prevents shared cells, and value p saturates every start arc. Conversely, routing one unit along each vertex-disjoint escape path respects all capacities and conservation.

For k=1 with the cell open, the boundary count is 1, not 4k−4=0. One person follows s→v_in→v_out→t for value 1; no people gives 0. Any boundary start can exit through its own internal edge without moving to a neighbor. A blocked cell cannot be a start. Rules allowing shared starts or exits require corresponding capacity and proof changes.

Maximum flow: solutions (continued)

26-E11 Reverse rerouting and a six-vertex cut certificate

Use original-flow vector order (sw,wx,xy,xt,sy,yz,zt):

Round	Path residual capacities before augmentation	δ	Original flows afterward	Value
1	(8,9,3,7,8)	3	(3,3,3,0,0,3,3)	3
2	(10,3,4)	3	(3,3,0,3,3,3,3)	6
3	(7,4,5)	4	(3,3,0,3,7,7,7)	10
4	(5,6,1)	1	(4,4,0,4,7,7,7)	11

Round 1 raises residual y→x from 0 to 3. Round 2 consumes it, canceling the old xy flow while increasing sy and xt by 3. Flows on sw,wx,yz,zt stay unchanged, preserving x,y balances. Initially c(y,x)=f(y,x)=0, so residual y→x is zero; the proposed initial s-y-x-t route is not a residual path.

Final positive residual arcs are sw4, ws4, wx5, xw4, xy3, sy3, ys7, tx4, zy7, zt1, tz7. From s the reachable set is exactly {s,w,x,y}. Residual xy is remaining original capacity, not the consumed reverse arc yx. The only original outgoing cut arcs are xt:4 and yz:7, totaling 11. Vertices w,x balance at 4 and y,z at 7. Source net outflow and sink net inflow are both 4+7=11, equal to the cut, proving optimality.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Maximum flow: solutions (continued)

Verification code appendix: flow_revision.py

```
"""Sparse exact net-flow models. Traces count operations, not measured speed."""
from collections import deque

class Network:
    def __init__(self,vertices,capacities,source,sink,neighbor_order=None):
        self.vertices=list(vertices);self.s=source;self.t=sink
        self.c={edge:value for edge,value in capacities.items() if value>0};self.f={}
        self.neighbors={u:[] for u in self.vertices}
        for u,v in self.c:
            if v not in self.neighbors[u]:self.neighbors[u].append(v)
            if u not in self.neighbors[v]:self.neighbors[v].append(u)
        if neighbor_order:
            for u,order in neighbor_order.items():
                if set(order)!=set(self.neighbors[u]):raise ValueError('neighbor order must
contain exactly original/reverse neighbors')
                self.neighbors[u]=list(order)
        self.e={u:0 for u in self.vertices};self.h={u:0 for u in self.vertices};self.current=
{u:0 for u in self.vertices}

    def residual(self,u,v):return self.c.get((u,v),0)-self.f.get((u,v),0)
    def send(self,u,v,amount):
        assert 0<amount<=self.residual(u,v)
        self.f[u,v]=self.f.get((u,v),0)+amount;self.f[v,u]=self.f.get((v,u),0)-amount
        self.e[u]-=amount;self.e[v]+=amount
    def value(self):return -self.e[self.s]
    def check(self,preflow=False,height=False):
        for u in self.vertices:
            assert self.e[u]==sum(self.f.get((v,u),0) for v in self.neighbors[u])
            if u!=self.s:assert self.e[u]>=0 if preflow else (u==self.t or self.e[u]==0)
            for v in self.neighbors[u]:
                assert self.f.get((u,v),0)==-self.f.get((v,u),0)
                assert self.f.get((u,v),0)<=self.c.get((u,v),0)
                if height and self.residual(u,v)>0:assert self.h[u]<=self.h[v]+1
        assert sum(self.e.values())==0
        if height:assert self.h[self.s]==len(self.vertices) and self.h[self.t]==0
    def reachable(self):
        seen={self.s};q=deque([self.s])
        while q:
            u=q.popleft()
            for v in self.neighbors[u]:
                if v not in seen and self.residual(u,v)>0:seen.add(v);q.append(v)
        return seen
    def cut_capacity(self,side):return sum(c for (u,v),c in self.c.items() if u in side and v
not in side)

def edmonds_karp(net):
    trace=[]
    while True:
```

Maximum flow: solutions (continued)

```
        parent={net.s:None};distance={net.s:0};q=deque([net.s])
        while q:
            u=q.popleft()
            for v in net.neighbors[u]:
                if v not in parent and
net.residual(u,v)>0:parent[v]=u;distance[v]=distance[u]+1;q.append(v)
        if net.t not in parent:return net.value(),trace
        path=[];v=net.t
        while v!=net.s:u=parent[v];path.append((u,v));v=u
        path.reverse();d=min(net.residual(u,v) for u,v in path)
        for u,v in path:net.send(u,v,d)

net.check();trace.append({'path':path,'delta':d,'distance':distance,'flow':dict(net.f)})

def relabel_to_front(net,trace_enabled=False):
    n=len(net.vertices);net.h[net.s]=n
    for v in net.neighbors[net.s]:
        amount=net.c.get((net.s,v),0)
        if amount:net.send(net.s,v,amount)
    linked=[u for u in net.vertices if u not in (net.s,net.t)];index=0;trace=[]
    def snapshot(action,u=None,v=None,amount=None):
        if
trace_enabled:trace.append({'action':action,'u':u,'v':v,'amount':amount,'e':dict(net.e),'h':d
ict(net.h),'current':{x:(net.neighbors[x][net.current[x]] if net.current[x]
<len(net.neighbors[x]) else None) for x in linked},'list':list(linked)})
    net.check(True,True);snapshot('initialize')
    while index<len(linked):
        u=linked[index];old=net.h[u]
        while net.e[u]>0:
            if net.current[u]==len(net.neighbors[u]):
                net.h[u]=1+min(net.h[v] for v in net.neighbors[u] if net.residual(u,v)>0)
                net.current[u]=0;net.check(True,True);snapshot('relabel',u)
            else:
                v=net.neighbors[u][net.current[u]]
                if net.residual(u,v)>0 and net.h[u]==net.h[v]+1:

amount=min(net.e[u],net.residual(u,v));net.send(u,v,amount);net.check(True,True);snapshot('pu
sh',u,v,amount)
                else:net.current[u]+=1
        if net.h[u]>old:linked.remove(u);linked.insert(0,u);snapshot('move-front',u)
        index=linked.index(u)+1
    net.check(False,True);return net.value(),trace

CAPACITY={('s','a'):4,('s','b'):3,('s','c'):2,('a','b'):2,('a','t'):1,('b','c'):2,
('b','t'):4,('c','t'):5}
ORDER={'a':['b','t','s'],'b':['c','t','a','s'],'c':['t','b','s']}
if __name__=='__main__':
    network=Network(['s','a','b','c','t'],CAPACITY,'s','t',ORDER)
    value,trace=relabel_to_front(network,True)
    for row in trace:print(row)
    side=network.reachable();print('value',value,'cut',side,network.cut_capacity(side))
```

Maximum flow: solutions (continued)

Verification code appendix: relabel_to_front_trace.py

```
"""Run the standard all-vertices relabel-to-front schedule on a small network."""

from dataclasses import dataclass, field


VERTICES = ("s", "a", "b", "c", "t")
CAPACITY = {
    ("s", "a"): 4,
    ("s", "b"): 3,
    ("s", "c"): 2,
    ("a", "b"): 2,
    ("a", "t"): 1,
    ("b", "c"): 2,
    ("b", "t"): 4,
    ("c", "t"): 5,
}
NEIGHBORS = {
    "a": ("b", "t", "s"),
    "b": ("c", "t", "a", "s"),
    "c": ("t", "b", "s"),
}


@dataclass
class Vertex:
    name: str
    height: int = 0
    excess: int = 0
    current: int = 0
    flow: dict = field(default_factory=dict)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Maximum flow: solutions (continued)

```
@dataclass
class Network:
    source: Vertex
    sink: Vertex
    vertices: dict

    def net_flow(self, u, v):
        return self.vertices[u].flow.get(v, 0)

    def residual(self, u, v):
        return CAPACITY.get((u, v), 0) - self.net_flow(u, v)

    def push(self, u, v):
        amount = min(self.vertices[u].excess, self.residual(u, v))
        assert amount > 0
        self.vertices[u].flow[v] = self.net_flow(u, v) + amount
        self.vertices[v].flow[u] = self.net_flow(v, u) - amount
        self.vertices[u].excess -= amount
        self.vertices[v].excess += amount


def state(net):
    return tuple(net.vertices[name].excess for name in ("a", "b", "c", "t"))


def heights(net):
    return tuple(net.vertices[name].height for name in ("a", "b", "c"))
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Maximum flow: solutions (continued)

```
def discharge(net, name):
    u = net.vertices[name]
    old_height = u.height
    rows = []
    while u.excess > 0:
        neighbors = NEIGHBORS[name]
        if u.current == len(neighbors):
            u.height = 1 + min(
                net.vertices[v].height
                for v in neighbors
                if net.residual(name, v) > 0
            )
            u.current = 0
            rows.append(("relabel", state(net), heights(net)))
            continue
        v = neighbors[u.current]
        if net.residual(name, v) > 0 and u.height == net.vertices[v].height + 1:
            net.push(name, v)
            rows.append((f"push->{v}", state(net), heights(net)))
        else:
            u.current += 1
    return u.height > old_height, rows


def run():
    vertices = {name: Vertex(name) for name in VERTICES}
    vertices["s"].height = len(VERTICES)
    net = Network(vertices["s"], vertices["t"], vertices)
    for target, capacity in (("a", 4), ("b", 3), ("c", 2)):
        vertices["s"].flow[target] = capacity
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Maximum flow: solutions (continued)

```
        vertices[target].flow["s"] = -capacity
        vertices[target].excess += capacity
        vertices["s"].excess -= capacity

    assert state(net) == (4, 3, 2, 0)
    assert heights(net) == (0, 0, 0)
    linked = ["a", "b", "c"]
    index = 0
    while index < len(linked):
        name = linked[index]
        increased, _ = discharge(net, name)
        if increased:
            linked.remove(name)
            linked.insert(0, name)
        index = linked.index(name) + 1

    assert linked == ["c", "b", "a"]
    assert state(net) == (0, 0, 0, 8)
    assert heights(net) == (6, 1, 1)
    assert sum(net.net_flow("s", v) for v in ("a", "b", "c")) == 8
    assert vertices["s"].excess == -8
    assert sum(v.excess for v in vertices.values()) == 0
    print("PASS relabel-to-front list=c,b,a excess=(0,0,0,8) heights=(6,1,1)")
    return linked, state(net)


if __name__ == "__main__":
    run()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study